

Ringkasan Per Role — YOLO Bukukia Training Pipeline

Ringkasan 1 Menit (untuk semua orang)
Untuk Product Manager & Product Design
Untuk UI/UX
Untuk Frontend Engineer & Backend Engineer
Untuk Data Engineer
Untuk AI Engineer / ML Engineer
Untuk Security
Untuk DevOps

Ringkasan Per Role — YOLO Bukukia Training Pipeline

Sistem ini menjelaskan pipeline training model deteksi untuk Buku KIA (Kartu Identitas Anak), ditulis lintas peran: PM, PD, UI/UX, FE, BE, Data Engineer, AI Engineer, Security, DevOps. Detail teknis baris kode ada di [../02-technical/ARSITEKTUR-DAN-SCRIPT.md](#).

Ringkasan 1 Menit (untuk semua orang)

Sistem ini bukan aplikasi yang dipakai langsung oleh pengguna. Ini adalah pipeline kerja untuk melatih model deteksi. Prosesnya:

1. Kita punya foto-foto Buku KIA yang sudah ditandai (dikasih kotak) di bagian-bagian pentingnya — misalnya kotak untuk NIK, kotak untuk nama, dst.
2. Model dilatih dari contoh-contoh itu supaya bisa menebak sendiri di mana letak kotak-kotak itu pada foto Buku KIA baru yang belum pernah dilihat.
3. Model yang sudah jadi (`best.pt`) lalu dipakai oleh sistem OCR lain (project terpisah) untuk memotong bagian foto yang relevan sebelum dibaca teksnya.

Bayangkan proses ini seperti mengajarkan petugas menemukan kolom NIK pada formulir melalui banyak contoh yang sudah diberi tanda. Setelah mendapat cukup contoh, model dapat mencari area yang sama pada gambar baru.

Yang perlu diketahui semua role: ini bagian dari pipeline OCR KIA, tapi berjalan terpisah dan tidak real-time — training butuh waktu (bisa puluhan menit–jam) dan dijalankan manual oleh tim AI/data. Hasilnya adalah satu file model (`best.pt`) yang di-deploy ke sistem OCR produksi. Datanya berisi data pribadi (nama orang tua, NIK anak) — lihat bagian Security di bawah.

Untuk Product Manager & Product Design

Apa yang sistem ini lakukan untuk produk? Sistem ini menentukan seberapa akurat OCR bisa menemukan "di mana" letak informasi penting di foto Buku KIA (nama, NIK, dst) sebelum dibaca. Kalau model ini salah menebak lokasi, hasil OCR bisa salah baca meski teksnya jelas — karena dia baca area yang salah.

Kualitas model diukur dengan:

- mAP50 / mAP50-95 — skor akurasi standar industri untuk deteksi objek (makin tinggi makin baik, target umum ≥ 0.8 dianggap "bagus" oleh sistem ini sendiri).
- Precision/Recall/F1 per kelas — seberapa sering model benar menandai tiap jenis field (NIK, nama, dst), dan seberapa sering dia melewatkan atau salah tandai.
- Ada laporan visual otomatis (`training_analysis.png`, `test_dashboard.png`) yang bisa diminta tim AI Engineer sebagai bukti kualitas sebelum model dipakai produksi.

Batasan yang perlu PM tahu:

- Dataset training saat ini kecil (91 foto aktif) — sistem sendiri punya alarm otomatis "dataset kecil" kalau di bawah 100 foto. Ini artinya akurasi model sangat bergantung pada jumlah & keragaman data yang di-label. Kalau mau tingkatkan akurasi, prioritas nomor satu biasanya tambah data berlabel, bukan oprek parameter.
- Proses menambah data baru (dari batch KIA baru) → training ulang → evaluasi → deploy adalah siklus manual, belum otomatis (tidak ada CI/CD untuk model). Perlu direncanakan sebagai task tersendiri tiap kali ada batch data baru.
- Ada beberapa bagian tooling yang belum rapi/belum lengkap (dicatat di bagian bawah) — kalau mau dipakai untuk fitur produk baru (misal auto-relabel dari feedback user), perlu review teknis dulu, jangan asumsikan semua fungsi siap pakai.

Tidak ada UI di sini — semua dijalankan lewat command line / notebook oleh tim teknis. Tidak relevan langsung untuk kerja UI/UX kecuali untuk memahami "kenapa OCR kadang salah baca field tertentu" (jawabannya sering ada di kualitas model & dataset di sini, bukan di aplikasi OCR-nya sendiri).

Untuk UI/UX

Tidak ada antarmuka pengguna di project ini — murni pipeline backend/data science. Yang relevan untuk kalian: kalau ada laporan user "OCR salah baca kolom X di Buku KIA", akar masalahnya bisa jadi di sini (model belum pernah lihat contoh field seperti itu, atau posisi kotaknya kurang presisi) — bukan selalu di alur UI OCR. Berguna untuk triase bug bersama tim AI Engineer.

Untuk Frontend Engineer & Backend Engineer

Project ini tidak punya API atau service yang jalan terus — tidak ada endpoint untuk dipanggil. Interaksinya satu arah: model (`best.pt`) yang dihasilkan di sini di-copy/dideploy ke service OCR lain (project terpisah, di luar folder ini), di sanalah FE/BE berinteraksi dengannya lewat API OCR yang sudah ada.

Yang perlu diketahui:

- Kalau ada perubahan daftar field yang dideteksi (`classes.txt` — saat ini 6 kelas: 5 segmen + NIK), itu akan mengubah kontrak apa saja yang bisa dikembalikan OCR ke aplikasi. Perubahan ini harus dikoordinasikan dengan tim yang pegang service OCR & format response API-nya.
 - Update model tidak otomatis ter-deploy — perlu ada proses manual/CI terpisah untuk menyalin `best.pt` baru ke environment produksi OCR. Kalau belum ada, ini gap yang perlu dikoordinasikan dengan DevOps.
-

Untuk Data Engineer

Sumber data & alur data:

- Data mentah (foto + label) berasal dari proses labeling (kemungkinan Label Studio, berdasarkan pola nama file `task19268...` yang mirip task ID Label Studio) dan sebagian diambil dari Google Cloud Storage (lihat `input_files_kia2024/export/gcs_manifest.txt` — bucket sumber: `test-kia-legacy-cover`).
- Ada dua folder dataset: `input_files/` (dataset aktif, 91 foto + label) dan `input_files_kia2024/` (hanya label, tanpa foto — arsip batch KIA 2024 lama yang foto mentahnya belum/tidak ada di repo lokal). Ini bukan dua versi dataset yang berbeda strukturnya, tapi label dari batch berbeda yang mungkin perlu digabung ulang dengan fotonya dari GCS kalau mau dipakai untuk training.
- Format label: YOLO standar (koordinat kotak ternormalisasi 0–1 per baris teks), 1 file `.txt` per foto.
- Split data train/val dilakukan otomatis tiap kali training (80/20, random seed tetap supaya konsisten) — bukan split yang disimpan permanen kecuali pakai opsi split manual.

Yang perlu diwaspadai:

- Ada script filtering data (`filter_labels_by_segment_colab.py`) yang memakai daftar ID kelas lama/tidak sinkron dengan `classes.txt` yang dipakai sekarang (lihat detail di dokumen teknis §7 & §9). Jangan pakai script ini untuk pipeline data baru tanpa verifikasi ulang mapping-nya dulu — risikonya salah label kalau dipakai apa adanya.
 - Tidak ada validasi otomatis kualitas data masuk (misal cek kotak label valid, tidak ada file orphan) — kalau proses ingest data baru mau dibuat lebih otomatis, ini area yang perlu ditambah.
-

Untuk AI Engineer / ML Engineer

Ini bagian paling relevan untuk kalian — detail penuh alur training, evaluasi, dan tuning ada di [dokumen teknis](#). Ringkasannya:

- Base model: YOLO (`yolo26n.pt` sebagai default, model kecil/nano — cocok untuk cepat tapi kapasitasnya terbatas, ada saran otomatis upgrade ke varian lebih besar kalau terindikasi underfitting).

- Loss function pakai Focal Loss by default (untuk menangani ketidakseimbangan kelas), bisa dimatikan lewat parameter.
 - Ada tracking Mean IoU per epoch, tapi cara hitungnya ada fallback estimasi kasar kalau data asli dari validator tidak tersedia — jangan anggap angka IoU di log selalu presisi, cek dulu apakah dia hasil fallback atau bukan.
 - Dua jalur fine-tuning tersedia (otomatis via Ultralytics `model.tune()`, atau manual coordinate-descent) — pilih salah satu sesuai kebutuhan, keduanya independen.
 - `run_pipeline.py` bukan orchestrator penuh — dia tidak menjalankan training, hanya evaluasi + fine-tune opsional + generate laporan HTML. Training (`train_2.py`) harus dijalankan manual duluan.
 - Ada beberapa script evaluasi dengan nama mirip (`evaluate_iou_train.py`, `evaluate_iou_full_train.py`, `evaluate_iou_full.py`) — fungsinya berbeda tipis, lihat tabel perbandingan di dokumen teknis supaya tidak salah pakai.
 - Ada satu potensi bug di `run_pipeline.py` terkait argumen CLI yang diteruskan ke `fine_tune.py` — dicek dulu sebelum dipakai dengan flag `--fine-tune` (detail di dokumen teknis §5).
-

Untuk Security

Data sensitif yang ada di project ini:

- Folder `input_files/raw_images/` dan `input_files_kia2024/` berisi foto asli dokumen kependudukan — nama ibu, nama ayah, NIK anak, alamat (lingkungan/kelurahan/kecamatan) — semuanya bisa dibaca langsung dari nama file, bukan cuma isi fotonya. Contoh nama file: `task19268_1003_KIA_SUAIDAH_RODI ZULKARNAEN_7000158101_SEPIT_SEPIT_KERUAK_page1.png` — ini sudah membocorkan nama & NIK di level nama file, tanpa perlu buka gambar.
- File `gcs_manifest.txt` mencatat path asal data di bucket GCS (`test-kia-legacy-cover`) — kalau bucket ini bukan bucket yang seharusnya diakses publik, pastikan aksesnya tetap private.
- Ini adalah PII (Personally Identifiable Information) penuh, bukan data anonim/sample. Perlu dipastikan:
 - Folder ini tidak ter-commit ke repo publik. **Update Agustus 2026:** repo ini sebelumnya tidak punya `.gitignore` sama sekali — sudah ditambahkan (lihat `root.gitignore`) yang mengecualikan `input_files/`, `input_files_kia2024/`, `results/`, dan `best.pt`. Sebelum perbaikan ini, folder-folder itu berisiko ter-git add tidak sengaja meski belum ada bukti ter-commit ke history saat ditemukan.
 - Akses ke direktori project ini dibatasi ke tim yang memang perlu (data science/AI engineer), bukan semua orang.
 - Kalau data ini pernah/akan dipindah ke Colab (banyak script di sini ditujukan untuk Colab) — data PII akan ter-upload ke environment Google Colab pihak ketiga. Perlu kebijakan jelas: apakah ini diizinkan, dan apakah perlu di-anonimkan dulu.
- Model hasil training (`best.pt`) sendiri tidak berisi data mentah secara langsung (ini bobot neural network, bukan database), tapi tetap perlu treatment akses yang wajar karena dilatih dari data sensitif.

Rekomendasi: audit singkat — pastikan tidak ada foto/label KIA yang pernah ter-push ke remote git, dan review kebijakan pemakaian Colab untuk data ini.

Untuk DevOps

Infrastruktur yang dipakai saat ini:

- Training dijalankan di 3 environment berbeda tergantung skrip: lokal (macOS dengan MPS/Apple Silicon, atau Linux dengan CUDA/GPU), Google Colab, atau GCP VM (ada notebook khusus *_vm.ipynb). Deteksi environment otomatis di `config.py`.
- Tidak ada CI/CD untuk pipeline ini — semuanya dijalankan manual (command line atau notebook interaktif). Tidak ada containerization (Dockerfile) yang terlihat di struktur project.
- Dependency Python ada di `requirements.txt` — perlu dicek versi `ultralytics` dan kompatibilitasnya kalau mau distandarkan across environment.
- Output model (`best.pt`) saat ini disimpan lokal di folder project, bukan di model registry terpusat (tidak ada integrasi MLflow/model registry yang terlihat). Kalau mau ada proses deploy model ke produksi OCR yang lebih terstruktur, ini gap yang perlu diisi — saat ini kemungkinan proses copy manual file.
- Tidak ada monitoring/alerting otomatis untuk training run yang gagal — hanya laporan HTML lokal yang dibuka manual setelah `run_pipeline.py` selesai.

Rekomendasi kalau mau distandarkan: pertimbangkan model registry sederhana + versi model yang jelas (bukan cuma satu file `best.pt` yang ditimpa terus), dan pipeline CI ringan minimal untuk validasi (misal cek dataset ter-load benar) sebelum training panjang dijalankan.